

2.8 MADiC: A 3nm 7.4TOPS/mm², 17.4TOPS/W Generative Diffusion Accelerator Enabled by Hardware-Compiler Co-Optimization of Memory Hierarchy and Operator Parallelism

Shih-Wei Hsieh, Yi-Syuan Chen, Ping-Yuan Tsai, Ming-Hung Lin, Chia-Yuan Cheng, Lien-Feng Hsu, Po-Hao Huang, Hung-Wei Chih, Po-Han Chiang, Chia-Ming Chang, Ming-Hsuan Chiang, Chia-Hung Yuan, Sheng-Po Kuo, Viswanath Uggu, Chun-Kun Chan, Ming-En David Shih, Yu-Cheng Tseng, Hsin-Ping Cheng, Stan Huang, Chia-Ping Chen, ShenKai Chang, Chih-Ming Wang, Po-Yu Yeh, Jett Liu, Yung-Chang Chang, Chi-Cheng Ju, Yuchean Kevin Jou

MediaTek, Hsinchu, Taiwan

Abstract

This work presents MADiC, a 3nm 0.338mm² diffusion accelerator for generative image editing on edge devices. MADiC features design-time L1/L2 memory optimization, cross-loop feature map allocation with write-over-read relaxation, and activation-aware input

preloading with concurrent resizing. Operating at 0.575V and 546MHz, it delivers 7.4TOPS/mm² and 17.4TOPS/W when running the Diffusion ConvNet model.

Generative diffusion models have become the mainstream approach for photo-realistic image synthesis, delivering exceptional quality across a range of applications including super-resolution and style transfer [1-3]. However, a significant gap remains in translating these capabilities to edge devices, where compact area and limited power budgets are challenged by the substantial computational and bandwidth demands inherent to these models. Recent algorithmic advances have begun to address these challenges. Diffusion Transformers (DiT) [4] leverage transformer architectures for better scalability, while step distillation [5, 6] reduces denoising steps from hundreds to one, reducing complexity by 50-100 $\times$. Diffusion ConvNets (DiCo) [7, 8] and architectural distillation [9] eliminate costly attention operations and further reduce model complexity, achieving an additional 3-to-10 $\times$ reduction. These advances narrow the gap for diffusion model deployment; however, significant hardware challenges persist as DiCo-based models require multi-scale feature connections and squeeze-and-excitation (SE) mechanisms [10] in each block. Both induce large memory bandwidth burdens and increased latency on typical AI accelerators, resulting in low operational intensity and utilization for partial computation loops.

Increasing on-chip memory enables more data reuse, which improves operational intensity and alleviates memory bottlenecks in these loops. However, as technology moves to more advanced nodes, SRAM area scaling has increasingly lagged behind logic, as shown in Fig. 2.8.1. Our empirical study shows that from 28nm to 3nm, the SRAM area shrink at only 0.44 $\times$ the rate of logic. This discrepancy poses a significant challenge for efficiently allocating on-chip memory to reduce external memory access (EMA) in area-constrained edge devices. In addition to memory bottlenecks, hardware utilization drops when input data fetching and resizing are performed sequentially with convolution, causing idle hardware and limited performance. To address these challenges, we present MADiC, a 3nm mini accelerator for Diffusion ConvNets, featuring: 1) design-time L1/L2 memory size optimization for minimizing SRAM area cost at a fixed EMA target, 2) cross-loop feature map (FM) storage allocation with write-over-read relaxation to reduce EMA, and 3) activation-aware input preloading and concurrent resizing enable operator parallelism to improve utilization. As a result, MADiC delivers 7.4TOPS/mm² and 17.4TOPS/W when running the DiCo model.

Figure 2.8.2 shows the overall architecture of MADiC, which includes a tensor core, a DMA module, 128KB L2 and 384KB L1 memory for activations, instructions, and weights, a preload controller, and two data streaming modules – the cross-loop feature map (FM) streamer and the activation L1 interconnection – that bridge the memory system and the tensor core datapath. The tensor core comprises a convolution (Conv) core with 2304 MAC units, a vector unit for element-wise operations, and a resizer for scaling and mean operations. Each unit's input and output connect to either the dual-mode activation L1 or the cross-loop FM streamer via the activation interconnection. The dual-mode activation L1 provides either dedicated bandwidth for parallel access or increased capacity for single-client access. The cross-loop FM streamer manages activation data across computation loops and selectively localizes frequently reused activations in L2 memory. Control registers, instructions, and weights are configured by a binary file generated by the accompanying compiler. At design time, the compiler serves as a design space optimizer, evaluating architectural choices and optimizing on-chip memory size for the target model pool. Once the memory size is fixed, the compiler performs memory allocation optimization, binary file generation, and power and latency estimation at runtime.

Figure 2.8.3 illustrates the design space exploration workflow that optimizes on-chip memory size under a predefined EMA target. MADiC executes a network by dividing it into inner loops, each representing a fused stack of layers for depth-first execution. During each inner loop, temporary FMs are stored in activation L1, while cross-loop FMs are exchanged via L2 memory or DRAM. Increasing L1 enables deeper inner-loop fusion and reduces external memory exchanges, while increasing L2 allows more cross-loop activations to remain on-chip. Both strategies lower EMA, leading to an area allocation tradeoff between L1 and L2. Additionally, since the benefits of scaling L1 or L2 are non-linear and depend strongly on the model topology, operation types, and tensor dimensions, we perform design space exploration over a pool of models, including both the main target and other representative models, to avoid overfitting to specific workloads. The exploration evaluates

the weighted sum of EMA for a set of (L1, L2) combinations, where area is calculated as the sum of L1 and L2 sizes normalized by their respective memory densities. Each evaluated (L1, L2) pair is plotted on the EMA-area graph, with the Pareto front showing the most efficient options. The final design is chosen as the Pareto-optimal point with the smallest area that satisfies the EMA constraint, minimizing the costly SRAM area in advanced nodes with minimal power overhead. Unlike traditional cache hierarchies that favor smaller lower-level caches without model- or hardware-specific optimization, our compiler-driven, design-time flow analyzes hardware-specific reuse and compiler-identified fusion to jointly optimize L1 and L2 for the target model pool. This approach outperforms typical baseline choices in both area and power, with 11% less area and 27% lower EMA than a naive L1:L2=1:4 allocation.

Figure 2.8.4 shows our cross-loop FM allocation strategy, which maximizes L2 utilization by allocating FMs at a fine-grained, line-based level. Each FM is defined by its lifetime, line count, line size, and total accesses. The L2 bandwidth contribution of each FM is calculated as its allocated size times its access count. Our objective is to maximize total L2 bandwidth and minimize EMA, which can be formulated as an Integer Linear Programming (ILP) problem. This formulation includes two key constraints: the non-overlap constraint ensures that FMs with overlapping lifetimes are assigned to distinct addresses in L2, while the non-overflow constraint keeps the total allocation within the available L2 size. We solve the problem optimally using the CP-SAT solver [11]. While previous works [12–14] have primarily focused on finding feasible allocations, our formulation explicitly optimizes for bandwidth. To further reduce memory waste, we apply Write-over-Read (WoR) relaxation by analyzing lifetimes at the line level within each FM. Instead of assigning a single lifetime to each FM, we calculate a minimum line offset K_{ij} between FM_i and FM_j , which represents the smallest safe distance that allows their memory regions to partially overlap. K_{ij} is derived from the allocation and deallocation times of each line, and is integrated into the non-overlap constraint to enable tighter L2 memory usage. We design a light-weight cross-loop FM streamer to support the line-based allocation strategy. The streamer maintains a table for each FM with the base address, line size, and total allocated lines in L2. For each data access, it uses a counter table to track the current line count and computes the L2 address as (base + current line count $\times$ line size). If the current line count exceeds the allocated lines, the access is redirected to DRAM. We compare a greedy method, which heuristically places the FM with the highest access count first, to our optimal solution, both with and without WoR relaxation. The optimal solution without WoR improves L2 bandwidth by 21% over the greedy method, while enabling WoR relaxation provides an additional 15% improvement. From an area perspective, to achieve the same EMA target as the optimal method with WoR, the greedy method and the optimal method without WoR require 48KB and 16KB more L2 usage, respectively.

Figure 2.8.5 illustrates activation-aware input preloading and the concurrent resizing mechanism to improve hardware and L1 bandwidth utilization. In standard processing, input loading, convolution, and resizing are executed sequentially, resulting in underutilized compute units and memory bandwidth. To address this, input preloading initiates data transfer from DRAM to L1 at the earliest time possible, loading input data into L1 whenever write bandwidth is available. A preload sync signal is added to ensure that data dependencies are maintained. Since preloading extends the lifetime of input activations in L1, additional space is required for pre-allocation. Concurrent resizing further enables the resizer to operate in parallel with preloading and convolution. To support simultaneous access, L1 is partitioned into two banks, allowing the resizer and Conv core to operate without contention. The architecture supporting these features includes a preload controller and a dual-mode activation L1. The preload controller monitors write requests from the Conv core to L1 and arbitrates access between DMA and Conv core on a per-cycle basis. A global sync unit coordinates the preload operation and tensor core for correct sequencing. When concurrent resizing is disabled, the dual-mode activation L1 operates in stack mode, unifying two groups of activation SRAMs as a larger bank to increase overall capacity. During concurrent resizing, the groups operate independently to provide separate bandwidth for the resizer and Conv core. This design minimizes area overhead by reusing existing SRAM resources, eliminating the need for additional dedicated memory. The compiler enables input preloading and concurrent resizing by default, and selectively disables them only if L1 memory demand

exceeds available space. For the DiCo model, enabling input preloading and concurrent resizing reduces resizer time by 41% and input loading time by 45%.

Figure 2.8.6 presents the measurement results and comparison table. Fabricated in 3nm technology, MADiC occupies 0.338mm² with a total of 512KB on-chip memory. When evaluated on the DiCo model, MADiC achieves a generation energy of 0.011J/inference and a latency of 0.116s, enabling low energy consumption and fast inference for end-to-end diffusion-based image editing. Notably, running DiCo model on MADiC with 8b activations and weights maintains high output quality, with negligible difference compared to FP32 outputs. Operating at 0.575V, MADiC achieves an area efficiency of 7.4TOPS/mm² and an energy efficiency of 17.4TOPS/W, outperforming previous diffusion processors [15-17] that do not rely on inter-timestep redundancy in both metrics. Figure 2.8.7 shows the chip micrograph and measurement results for additional non-diffusion benchmark models, demonstrating MADiC's ability to sustain high efficiency across a wide range of AI workloads.

References:

- [1] L. Sun et al., "Pixel-Level and Semantic-Level Adjustable Super-Resolution: A Dual-Lora Approach," *IEEE CVPR*, pp. 2333-2343, 2025. <http://doi.org/10.1109/cvpr52734.2025.00223>
- [2] R. Xu et al., "StyleSSP: Sampling StartPoint Enhancement for Training-free Diffusion-based Method for Style Transfer," *IEEE CVPR*, pp. 18260-18269, 2025. <http://doi.org/10.1109/cvpr52734.2025.01702>
- [3] R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models," *IEEE CVPR*, pp. 10684-10695, 2022. <http://doi.org/10.1109/cvpr52688.2022.01042>
- [4] W. Peebles et al., "Scalable Diffusion Models with Transformers," *ICCV*, pp. 4195-4205, 2023. <http://doi.org/10.1109/iccv51070.2023.00387>
- [5] Y. Kim et al., "Autoregressive Distillation of Diffusion Transformers," *IEEE CVPR*, pp. 15745-15756, 2025. <http://doi.org/10.1109/cvpr52734.2025.01468>
- [6] T. Yin et al., "One-Step Diffusion with Distribution Matching Distillation," *IEEE CVPR*, pp. 6613-6623, 2024. <http://doi.org/10.1109/cvpr52733.2024.00632>
- [7] Y. Tian et al., "DiC: Rethinking Conv3x3 Designs in Diffusion Model," *IEEE CVPR*, pp. 2469-2478, 2025. <http://doi.org/10.1109/cvpr52734.2025.00236>
- [8] Y. Ai et al., "DiCo: Revitalizing ConvNets for Scalable and Efficient Diffusion Modeling," *arXiv:2505.11196*, 2025. <http://doi.org/10.48550/arXiv.2505.11196>
- [9] B. Chen et al., "Adversarial Diffusion Compression for Real-World Image Super-Resolution," *IEEE CVPR*, pp. 28208-28220, 2025. <http://doi.org/10.1109/cvpr52734.2025.02627>
- [10] J. Hu et al., "Squeeze-and-Excitation Networks," *IEEE CVPR*, pp. 7132-7141, 2018. <http://doi.org/10.1109/cvpr.2018.00745>
- [11] Google OR Tools: CP-SAT Solver, Accessed Sept. 2025. http://developers.google.com/optimization/cp/cp_solver/
- [12] M. Mass et al., "TelaMalloc: Efficient On-Chip Memory Allocation for Production Machine Learning Accelerators," *ACM ASPLOS*, pp. 127-137, 2023. <http://doi.org/10.1145/3567955.3567961>
- [13] M. D. Moffitt, "MiniMalloc: A Lightweight Memory Allocator for Hardware-Accelerated Machine Learning," *ACM ASPLOS*, pp. 238-252, 2023. <http://doi.org/10.1145/3623278.3624752>

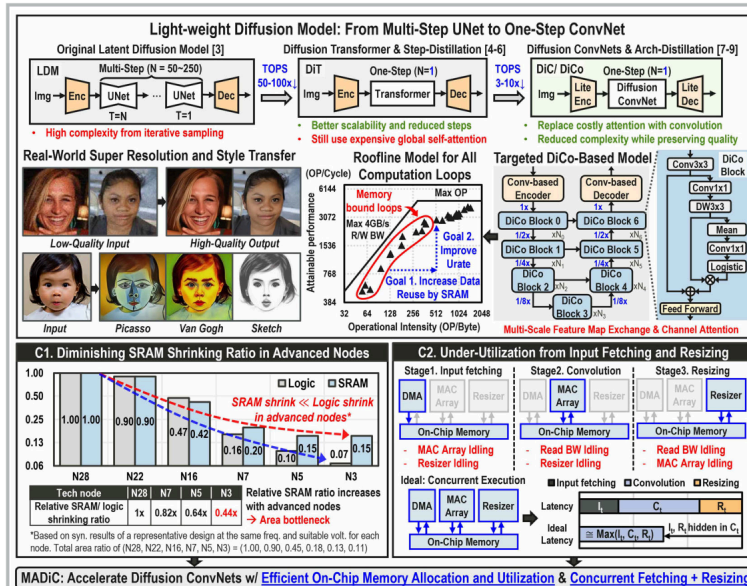


Figure 2.8.1: Overview of diffusion model evolution and challenges in area and utilization for efficient deployment.

- [14] M. Scherer et al., "DeepDeploy: Enabling Energy-Efficient Deployment of Small Language Models on Heterogeneous Microcontrollers," *IEEE TCAD*, pp.4009-4020, 2024. <http://doi.org/10.1109/tcad.2024.3443718>
- [15] S. Kim et al., "EdgeDiff: 418.4 mJ/Inference Multi-Modal Few-Step Diffusion Model Accelerator with Mixed-Precision and Reordered Group Quantization," *ISSCC*, pp.410-412, 2025. <http://doi.org/10.1109/isscc49661.2025.10904594>
- [16] D. Han et al., "MEGA-mini: A Universal Generative AI Processor with a New Big/Little Core Architecture for NPU," *ISSCC*, pp.416-418, 2025. <http://doi.org/10.1109/isscc49661.2025.10904514>
- [17] S-W. Hsieh et al., "MAE: A 3nm 0.168mm² 576MAC Mini AutoEncoder with Line-Based Depth-First Scheduling for Generative AI in Vision on Edge Devices," *ISSCC*, pp. 414-416, 2025. <http://doi.org/10.1109/isscc49661.2025.10904763>
- [18] M-E. Shih et al., "NVE: A 3nm 23.2 TOPS/W 12b-Digital-CIM-Based Neural Engine for High-Resolution Visual-Quality Enhancement on Smart Devices," *ISSCC*, pp. 360-362, 2024. <http://doi.org/10.1109/isscc49657.2024.10454482>
- [19] R. Guo et al., "A 28nm 74.34 TFLOPS/W BF16 Heterogenous CIM-Based Accelerator Exploiting Denoising-Similarity for Diffusion Models," *ISSCC*, pp. 362-364, 2024. <http://doi.org/10.1109/isscc49657.2024.10454308>
- [20] Y. Qin et al., "A 52.01 TFLOPS/W Diffusion Model Processor with Inter-Time-Step Convolution-Attention-Redundancy Elimination and Bipolar Floating-Point Multiplication," *IEEE Symp. VLSI Circuits*, pp. C304-C305, 2024. <http://doi.org/10.1109/vlsitechnologyandcir46783.2024.10631322>
- [21] J-S. Park et al., "An On-Device Generative AI Focused Neural Processing Unit in 4nm Flagship Mobile Soc with Fan-Out Wafer-Level Package," *ISSCC*, pp. 286-288, 2025. <http://doi.org/10.1109/isscc49661.2025.10904722>

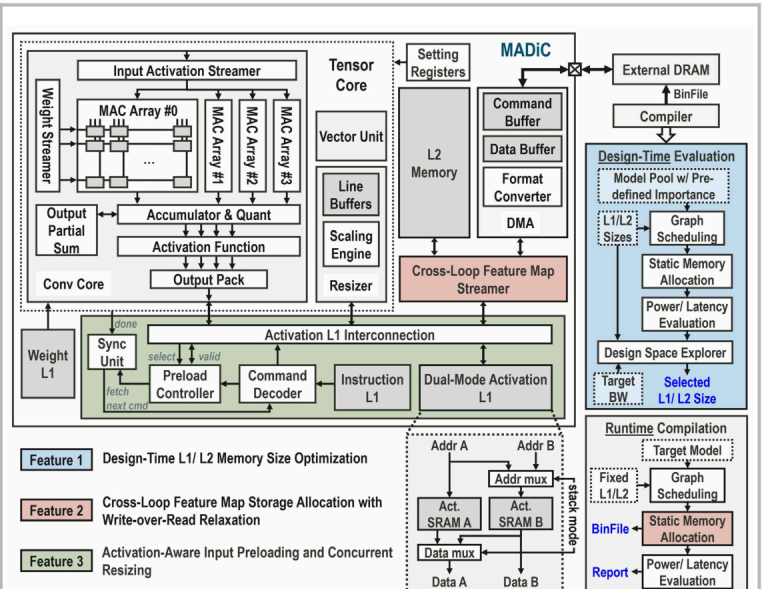


Figure 2.8.2: Overall architecture of MADiC.

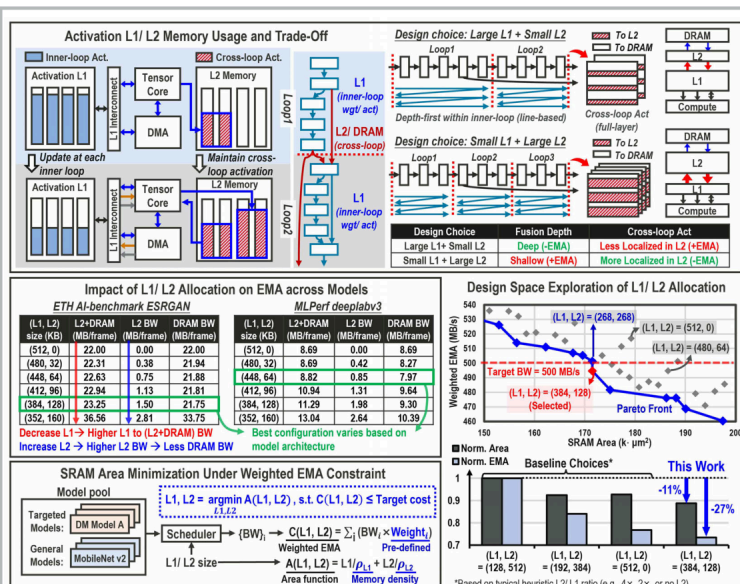


Figure 2.8.3: Design-time L1/L2 memory size optimization for minimizing SRAM cost.

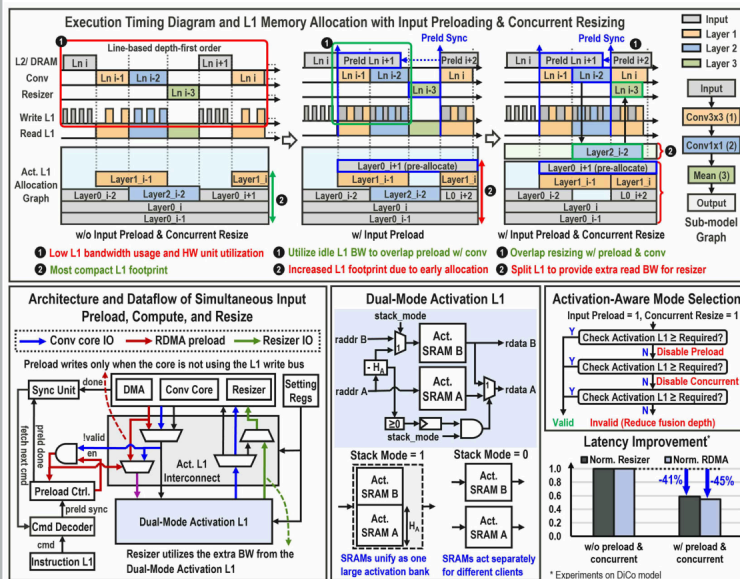


Figure 2.8.5: Activation-aware input preloading and concurrent resizing.

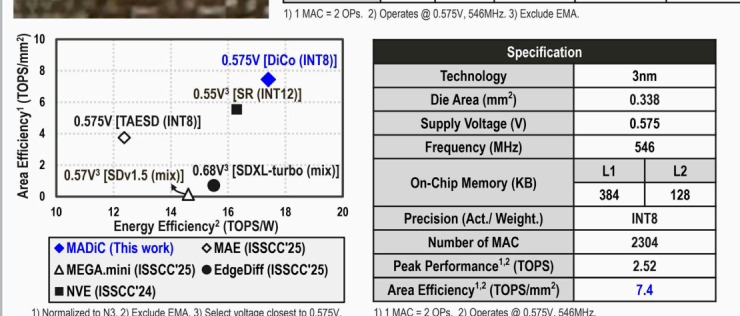
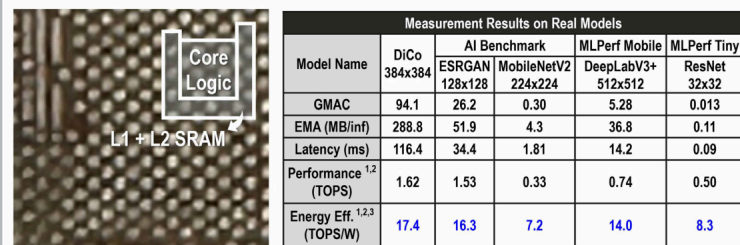


Figure 2.8.7: Chip micrograph and performance summary.

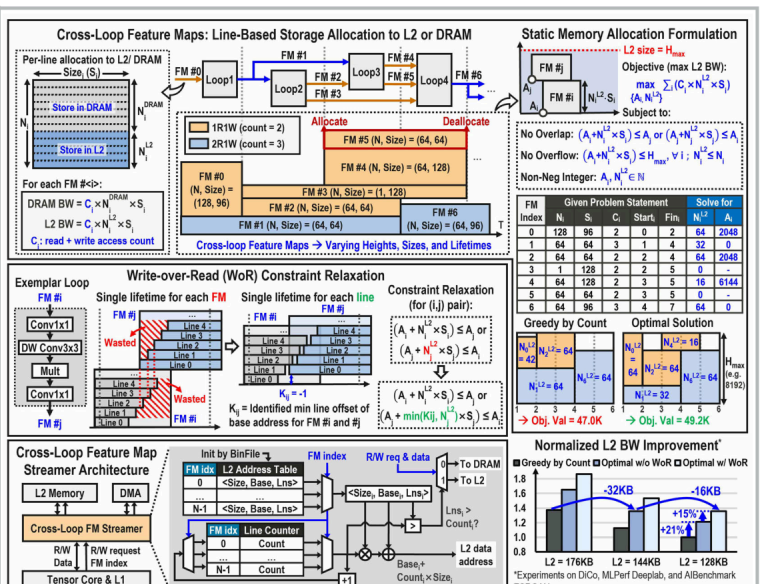


Figure 2.8.4: Cross-loop data storage allocation and Write-over-Read (WoR) relaxation.

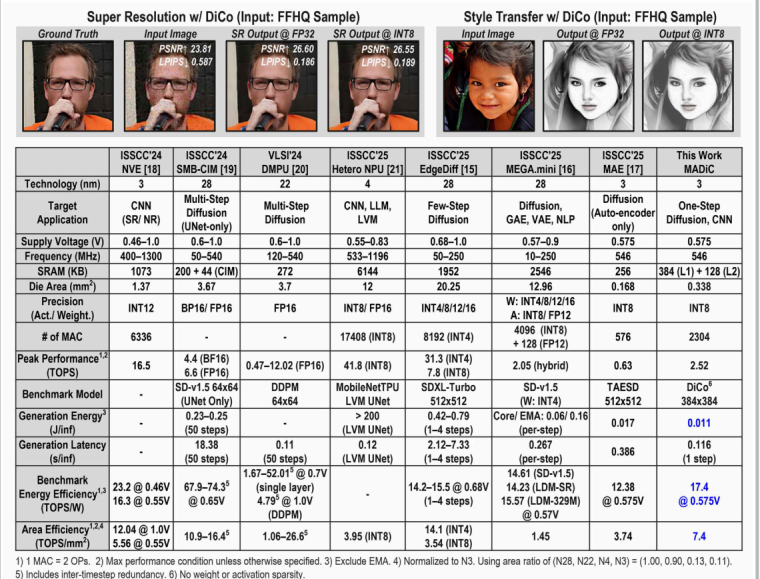


Figure 2.8.6: Measurement results and comparison table.